

Pattern Singleton en C#

Mauvaise pratique (en environnement multi threads)

```
using System;

public class Singleton
{
    private static Singleton instance;

    private Singleton() {}

    public static Singleton Instance
    {
        get
        {
            if (instance == null)
            {
                instance = new Singleton();
            }
            return instance;
        }
    }
}
```

Bonne pratique (en environnement multi threads)

```
using System;

public sealed class Singleton
{
    private static volatile Singleton instance;
    private static object syncRoot = new Object();

    private Singleton() {}

    public static Singleton Instance
    {
        get
        {
            if (instance == null)
            {
                lock (syncRoot)
                {
                    if (instance == null)
                        instance = new Singleton();
                }
            }

            return instance;
        }
    }
}
```

sealed : Lorsqu'il est appliqué à une classe, le modificateur sealed empêche les autres classes d'en hériter

volatile : Le mot clé volatile indique qu'un champ peut être modifié par plusieurs threads qui s'exécutent simultanément. Les champs qui sont déclarés volatile ne sont pas soumis aux optimisations du compilateur qui supposent l'accès par un seul thread. Cela garantit que la valeur la plus à jour est présente dans le champ à tout moment.

Lock : Le mot clé lock marque un bloc d'instructions comme section critique en assurant le verrouillage par exclusion mutuelle d'un objet particulier, en exécutant une instruction, puis en annulant le verrouillage. Le mot clé lock permet de garantir qu'un thread n'entre pas dans une section critique de code pendant qu'un autre thread est dans la section critique. Si un autre thread tente d'entrer dans un code verrouillé, il attendra, bloquera, jusqu'à ce que l'objet soit libéré. La méthode conseillée consiste à définir un objet private à verrouiller.